Buffer Prospector: Discovering and Exploiting Untapped Buffer Resources in Many-Core DNN Accelerators

Yuchen Wei[†*], Jingwei Cai[†*], Mingyu Gao[†], Sen Peng[¶], Zuotong Wu[¶], Guiming Shi[†], Kaisheng Ma[†§]
[†] Tsinghua Univeristy, [¶] Xi'an Jiaotong University
[*] *Co-first Author,* [§] *Corresponding Author*
{weiyc22, caijw21}@mails.tsinghua.edu.cn, kaisheng@tsinghua.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132440

***Abstract*—In large-scale DNN inference accelerators, the many-core architecture has emerged as a predominant design, with layer-pipeline (LP) mapping being a mainstream mapping approach. However, our experimental findings and theoretical justifications uncover a hardware-independent and prevalent flaw in employing layer-pipeline mapping on many-core accelerators: a significant underutilization of buffer space across numerous cores, indicating substantial potential for optimization. Building on this discovery, we develop a universal and efficient buffer allocation strategy, BufferProspector, which includes a Buffer Requirement Calculator and Buffer Allocator, to capitalize on these unused buffers, addressing the timing mismatch challenge inherent in LP mapping. Compared to the state-of-the-art (SOTA) open-source LP mapping framework Tangram, BufferProspector averages a simultaneous increase in energy efficiency and performance by 1.44× and 2.26×, respectively. Moreover, we conduct some case studies on architecture and mapping. *BufferProspector will be open-sourced*.**

***Index Terms*—DNN Accelerators, Many-core, Buffer Management, Scheduling**

I. Introduction

Deep neural networks (DNNs) are widely deployed to solve real-world problems, such as image recognition [1], [2], [3] and natural language processing [4]. To achieve higher accuracy and robustness in complex scenarios, the size and complexity of DNN are becoming larger and larger, which results in increasing compute and storage demands. Therefore, many large-scale accelerators have been proposed to meet these demands [5], [6], [7], [8]. Among them, many-core architecture stands out as a predominant type [5], [6], [7], [8]. Within such accelerators, each core can independently handle a specific computational task. Moreover, the cores are interconnected via a Network-on-chip (NoC), ensuring mutual access.

Despite the rich on-chip computing and storage resources of large-scale many-core accelerators, harnessing their full potential to boost computing performance and energy efficiency remains challenging [9], [10], [11]. This has led to the introduction of LP mapping [8], [10], [11], [12]. In LP mapping, distinct layers are assigned to specific core groups, and their computed output feature maps (ofmaps) can be directly relayed to the consuming layer via the NoC, avoiding the expensive DRAM access costs. Compared to the traditional layer-sequential (LS) mapping [13], [14], where all computing cores are employed to compute layers one by one, LP mapping notably elevates both performance and utilization [11], [8]. For example, simply implementing LP mapping in Simba resulted in a 2.3× throughput increase over LS [8]. Due to its effectiveness, LP mapping is now widely embraced by most contemporary many-core accelerators [5], [6], [7], [8], [12].

Nonetheless, the prevailing LP mapping strategy exhibits a fundamental shortcoming: numerous cores suffer from extremely low buffer utilization rates. For example, when mapping ResNet-50 [1] on the default hardware setting (See Sec. V-A1) by existing SOTA LP mapping strategies [11], the buffer utilization of a half of the cores will be less than 25.64%. We conduct extensive theoretical analyses combined with empirical experiments to discern the direct proportional relationship between the Data-Compute Ratio (DCR) of each layer under LP mapping and the buffer utilization of the core processing it. DCR is defined as the ratio of a layer's data amount to its computation amount. Consequently, the substantial variability in DCR across different layers in modern DNNs contributes to the low utilization of numerous cores.

Upon unearthing this deficiency, we identified a significant amount of unused buffer. The next question to answer is: how to fully utilize such unused buffer resources and improve the performance and energy efficiency of the accelerator? We focus on the **timing mismatch challenge** in LP: Contemporary DNN architectures typically manifest as complex Directed Acyclic Graphs (DAGs) [1], [2], [4], and when two paths from a source layer to a destination layer have different lengths (e.g., in a residue block [1]), the shorter one will produce its ofmaps much earlier than the other. If we sent these early-produced fmaps directly to the consuming layer, there will be severe pipeline stalls. Thus, SOTA scheduling frameworks like Tangram [11] simply forbid such reuse and send it directly to DRAM for future usage, missing significant reuse opportunities. However, with the observation above, we can store such early-produced fmaps into the untapped buffers, which previous frameworks are unaware of, until the latest ones are ready.

Realizing the objectives above in a universal and efficient manner poses significant challenges. Specifically, with an arbitrary complex DNN DAG structure, how do we ascertain the buffer needs for each edge? Furthermore, how do we optimally distribute buffer space to accommodate these requirements? To address these issues, we introduce a comprehensive and efficient buffer allocation strategy, BufferProspector, which encompasses a universal buffer requirement calculator capable of pinpointing all edges that lead to timing discrepancies in any DNN DAG and determining the buffer needs for each edge. Thus, based on these requirements and the remaining capacity of each on-chip buffer, our efficient buffer allocator judiciously allocates unused on-chip buffer space to each demanding edge in a NoC-friendly manner.

In conclusion, we make the following contributions:

(1) We observe a fundamental deficiency in the widely-used LP mapping: a low buffer utilization rate in many cores, and analyze its cause qualitatively and quantitatively.

(2) To fully leverage the discovered unused buffer, we propose a universal and efficient buffer allocation strategy, BufferProspector, to address the time mismatch challenge within LP mapping. BufferProspector is open-sourced at this link [15].

(3) We conduct experiments spanning diverse DNN architectures, accelerator scales, and batch sizes. These experiments conclusively show that BufferProspector enhances both performance and energy efficiency over Tangram [11].

(4) We conduct case studies to explore the impact of buffer size,

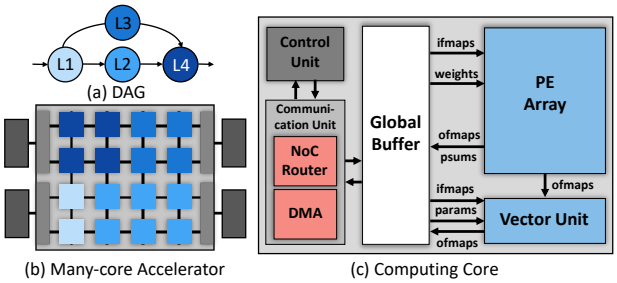


Fig. 1: Mapping Four Layers on A Many-core Accelerator.

a critical architectural factor, as well as the effect of pipeline depth, a key mapping factor, on performance and energy efficiency.

II. Background and Motivation

A. Many-core DNN Inference Accelerators

In recent years, many-core DNN inference accelerators have gained significant popularity in DNN acceleration. To facilitate a deeper investigation into their mapping strategies, we develop a universal and configurable hardware template (see Fig. 1) by extracting and synthesizing common features from a range of existing many-core inference accelerators [11], [5], [8], [10].

Overall, many-core accelerators comprise computing cores responsible for executing specific computing tasks, IO interfaces such as DRAM PHY & controllers (depicted as the light grey blocks on the edge of the chip in Fig. 1(b)), and a NoC, which manages data exchange between cores as well as between cores and DRAM.

The computing core (see Fig. 1(c)) is responsible for executing computing tasks. The following are its critical components. The communication unit, comprised of DMA and a router, enables seamless data exchange with other cores and DRAM. The control unit orchestrates computing tasks, drawing upon statically-compiled instructions and ongoing task progress data, and it also manages the inward and outward flow of data or messages from external cores or DRAMs. Each core is equipped with a Global Buffer (GLB), accessible throughout the accelerator, allowing for data reads and writes to and from the GLBs of other cores, given that the data is valid or the address is writable. The PE array and vector unit compute General Matrix Multiply (GEMM)/Convolution (Conv) operations and handle vector/scalar operations, respectively.

B. Layer-pipeline Mapping

In this section, we delve into the basis of LP Mapping and explore the challenges associated with timing mismatch that it faces.

1) Layer-pipeline Mapping: LP mapping is extensively employed in current large-scale many-core DNN accelerators [5], [8], [7], [6]. Under this mapping paradigm, each DNN graph is initially divided into various subgraphs, and each layer within a subgraph is assigned to a group of cores for computation. For instance, as illustrated in Fig. 1, L1-L4 are allocated 2, 6, 4, and 4 computing cores, respectively. Subsequently, a batch of data is divided into subbatches (SB), and each subbatch is processed in a pipelined manner by the core groups assigned to each layer. For example, once the first subbatch of data computed by L1 is sent to L2, L2 begins processing the first subbatch while L1 starts computing the second subbatch (see Fig. 2). This method can reduce the overhead of filling and draining while saving expensive DRAM access.

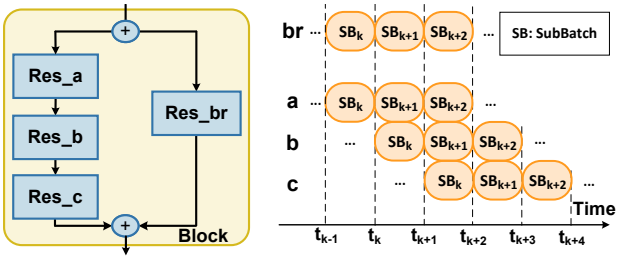


Fig. 2: An Example of Timimg Mismatch in LP Mapping.

2) Timimg Mismatch Challenge: With the development of Deep Learning, the DAG structures of DNN are becoming more and more complex [1], [16], [2], [4]. To balance each pipeline stage and reduce bubbles, the computing time of different Conv layers is kept roughly equal in LP mapping. This is achieved by controlling the number of cores allocated to each layer roughly proportional to the OP-num of that layer. However, when trying to pipeline map multiple branches with different lengths in a segment on the system, we will encounter timing mismatches. For example, as shown in Fig.2, both the shorter and longer branch receive the same input feature maps (ifmaps) at t_{k-1}, but the shorter one outputs data (at t_k) earlier than the longer branch output corresponding data (at t_{k+2}). However, the following element-wise add layer needs to get these ofmaps at the same time to calculate. This mismatch will lead to severe stalls in the pipeline, lowering the utilization of the system. Thus, existing works [11], [17], [18] give up mapping even one whole block containing multiple different branches, which limits the potential of LP mapping and increases expensive DRAM access costs.

III. Key Observation: Low Buffer Utilization with LP Mapping

As introduced in Sec. II-B, LP Mapping is widely employed by most many-core DNN accelerators due to its effectiveness in reducing expensive DRAM access and huge improvement on both performance and energy efficiency. However, in practice, we notice a key deficiency of current LP mapping on many-core accelerators: The buffer utilization of numerous cores is very low. For example, when mapping ResNet-50 [1] on the default hardware setting (See Sec. V-A1) by SOTA LP mapping framework [11], the buffer utilization of a half of the cores is less than 25.64%. Since buffers provide precious on-chip reuse opportunities, low buffer utilization implies that such opportunities are greatly wasted and the potential DRAM access reduction brought by on-chip data reuse in LP mapping is not fully explored.

Next, we aim to establish that this phenomenon is universally prevalent, solely related to the layers' DCR attribute, and independent of the hardware architecture. This proof also signifies the discovery of a substantial amount of underutilized buffer space, presenting numerous opportunities for optimization.

The Data-Compute-Ratio (DCR) of layer i is define as

$$DCR_i = \frac{data_i}{op_i} \tag{1}$$

where $data_i$ is the total data size of layer i, including ifmaps, weights, and ofmaps, and op_i is the OP-num of layer i. DCR depicts the ratio between the data size and computation of a layer. Typically, different layers in the same DNN will have different DCR values, and the variance of these values are relatively high. For example, the largest DCR among all convolution (Conv) layers in ResNet-50 [1] and DenseNet [2] are 11.25× and 6.14× larger than the smallest ones, respectively.

Then we conduct some theoretical analysis with some ideal assumptions to show the mathematical relation between DCR and buffer utilization of each layer. Let BU_i denote the buffer utilization of layer i, num_i denote the number of cores computing layer i, B denote the buffer capacity of each core, then we have

$$BU_i = \frac{data_i}{num_i \cdot B} \tag{2}$$

Furthermore, let C be the computing power of each core (the OP-num a core can compute in one time unit), t_i be the compute time of layer i, then by definition of C we have

$$t_i = \frac{op_i}{num_i \cdot C} \tag{3}$$

Combine equation 1, 2 and 3, we have

$$DCR_i = \frac{data_i}{op_i} = \frac{BU_i \cdot (num_i \cdot B)}{t_i \cdot (num_i \cdot C)} = \frac{B}{C} \cdot \frac{BU_i}{t_i} \tag{4}$$

which is a universal relation between DCR_i and BU_i: The DCR of a layer is proportional to the ratio between buffer utilization and compute time of that layer.

However, there is a fundamental property in LP mapping: To avoid bubbles and maintain high compute utilization, the compute time of each layer in the DNN must be roughly the same in LP mapping [8], [5], [10], [11]. In other words, there exists a constant T such that $t_i \approx T$ for different layers in the pipeline, then equation 4 becomes

$$DCR_i = \frac{B}{C} \cdot \frac{BU_i}{t_i} \approx \frac{B}{C \cdot T} \cdot BU_i = Const \cdot BU_i \tag{5}$$

This shows that the DCR of a layer is roughly proportional to its buffer utilization in LP mapping. Thus a high variance in DCR implies a high variance in the buffer utilization among different layers. Since the largest buffer utilization cannot exceed 100%, layers with low DCR will suffer from much lower buffer utilization.

Figure 3 demonstrates the DCR of four layers within a residual block from ResNet-50 and their corresponding buffer utilization in a practical LP mapping scenario. Among these layers, a relative proportionality is observed between their DCR and buffer utilization: Layer_a (L_a) has the highest DCR, which correlates with the largest buffer utilization among all layers. In contrast, layer L_b's DCR is only $0.089\times$ that of L_a, leading to a significantly lower buffer utilization of just 12.9%.

In summary, the analysis reveals a linear relationship between the DCR, a layer feature, and its buffer utilization. This, coupled with the fundamental requirement in existing layer-pipeline mapping strategies [11], [18], [17] for computational timing to be as identical as possible across different layers, leads to low buffer utilization under current mapping strategies. Thus, this deficiency is independent of hardware architecture and exhibits a universal presence.

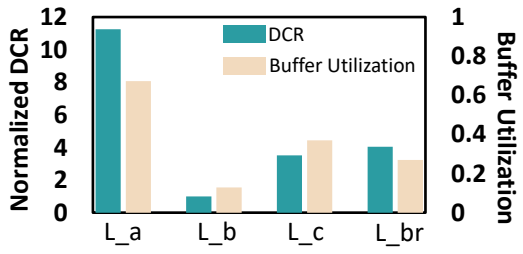


Fig. 3: DCR and Buffer Utilization of Mapping Four Layers in A Residual Block [1]. DCRs of all layers are normalized to the L_b layer with the lowest DCR.

IV. BufferPropector Strategy

After realizing the existence of such unused buffer on-chip, the next question to answer is: How can we turn such precious unused buffer resources into actual performance and energy efficiency gain? We focus on solving the timing mismatch challenge in LP mapping introduced in Sec. II-B2.

The high-level idea to solve the above challenge is straightforward: With the observation in Sec. III, we can store such early-produced fmaps (introduced in Sec. II-B2) into under-utilized buffers, which existing strategies are unaware of, and send buffered fmaps to the destination layer when required. As in the example in Sec. II-B2, the former ofmaps can be stored in unused buffers between t_k and t_{k+2}, and sent to the next layer at t_{k+2}. In this way, the ofmaps can be transmitted to the destination without storing into DRAM, which reduces a large amount of expensive DRAM accesses compared with existing works [11], [17], [18].

However, implementing this high-level concept universally and efficiently encounters two major challenges:

1. In an arbitrary DNN model with complex dependencies, how do we ascertain the buffer requirements for each timing mismatch?
2. With current buffer usage information, how do we efficiently allocate unused buffer space to accommodate these requirements?

Thus, we develop a universal and efficient buffer allocation strategy, BufferProspector, to solve these challenges. BufferProspector consists of a universal Buffer Requirement Calculator and an efficient Buffer Allocator. The Buffer Requirement Calculator detects all edges that lead to timing mismatch in the DNN dependency graph and calculates the buffer needed for each edge, solving the challenge 1. Then this calculated buffer requirement, along with the remaining capacity of each buffer, is sent to the Buffer Allocator, where the Buffer Allocator allocates unused on-chip buffer space to each edge in a NoC-friendly manner, which solves the challenge 2.

A. Buffer Requirement Calculator

Before calculating the buffer requirement of each edge in the DAG, we need to define the depth of each layer in the graph. For an arbitrary DNN with N layers denoted as $L_1, L_2, \ldots, L_N$, let $L_j \to L_i$ denote that there is an edge from L_j to L_i in the DAG. Then denote $P_i = \{L_j \mid L_j \to L_i\}$ be the preceding layers of L_i. Now the depth d_i of a layer L_i is defined as

$$d_i = \begin{cases} 0 & , P_i = \emptyset \\ \max_{L_j \in P_i} [d_j] + 1 & , P_i \neq \emptyset \end{cases} \tag{6}$$

where d_i can be calculated iteratively under a topological order of the DAG.

A layer has depth d means that there is a chain of d layers before that layer in the graph, and there does not exist any chain longer than d. Since when there is a dependency $L_j \to L_i$ in the graph, L_i must be processed at least one subbatch later than L_j (e.g. see Res_a $\to$ Res_b in Fig. 2), a layer of depth d will be processed d subbatches later than the first layer.

Then, for a dependency $L_j \to L_i$, the finishing of a subbatch at layer L_j is $d_i - (d_j + 1)$ subbatches before the starting of a subbatch at layer L_i. Thus if $d_i = d_j + 1$, then there is no need to buffer; if $d_i > (d_j + 1)$, a subbatch should be buffered during the processing of $d_i - (d_j + 1)$ subbatches, thus at most $d_i - (d_j + 1)$ subbatches will be buffered at the same time. However, to hide the latency of NoC transmission, each subbatch should be produced in a stream-processing manner and as long as some ofmaps are produced, NoC can start transmitting them. Similar to double buffering, such overlapping of NoC and processing requires the buffer capacity of

one additional subbatch, yielding a capacity of $d_i - d_j$ subbatches in total. Furthermore, notice that when the total number of subbatches NSB is smaller than $d_i - d_j$, then there is only NSB subbatches to be buffered in total. So if we denote c_j as the size of ofmaps of one subbatch in L_j, the capacity needed for $L_j \to L_i$ is

$$Cap_{j,i} = \min\left[d_i - d_j, NSB\right] \cdot c_j \tag{7}$$

Finally, when there are multiple edges starting from L_j that are needed to buffer, we only need to buffer the edge with the longest time interval. Since all buffering starts at the same time, the buffer needed for the ofmaps of L_j is:

$$Cap_j = \max_{\substack{L_j \to L_i \\ d_i > d_j + 1}} \left[Cap_{j,i}\right] = \min\left[\max_{\substack{L_j \to L_i \\ d_i > d_j + 1}} \left[d_i\right] - d_j, NSB\right] \cdot c_j \tag{8}$$

and if there are no such $L_j \to L_i$ with $d_i > d_j + 1$, let $Cap_j = 0$ and there is no need to buffer.

Taking Fig. 2 as an example: If the depth of the top Add layer is 0, then the depth of the bottom Add layer and Res_br is $d_{add} = 4$ and $d_{br} = 1$, respectively. Since SB_k is produced by Res_br before t_k and produced by the bottom add layer after t_{k+2}, it needed to be buffered between t_k and t_{k+2}. Then considering the transmission of NoC, we also need to allocate its buffer between t_{k-1} and t_k to receive the partial result from Res_br. Similarly, SB_{k+i} needs an allocation of buffer between t_{k-1+i} and t_{k+2+i}. In this way, the buffer needs to buffer SB_k, SB_{k+1}, SB_{k+2} at once, resulting with a capacity of 3 subbatches, which matches with $d_{add} - d_{br}$.

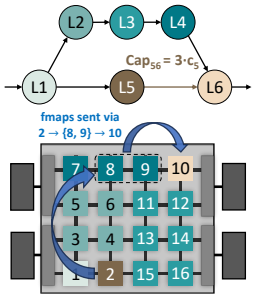
In summary, the Buffer Requirement Calculator firstly finds a topological order of the layers in the DNN, then calculate $d_1, d_2, \ldots, d_n$ using equation 6, at last outputs the buffer requirement of each layer by equation 8.

B. Buffer Allocator

After the buffer requirement of each edge is obtained, we need to allocate unused on-chip buffer space to satisfy these requirements. During the allocation, there are three factors to consider:

Number of Cores: Since the buffer requirement size usually contains multiple subbatches, we need to select multiple cores to buffer early-produced fmaps in many cases. However, allocating too many cores for a single requirement can lead to considerable concurrent synchronization overheads when the subsequent layer needs to fetch these fmaps.

Total NoC Distance: The NoC transmission of these fmaps will not only produce additional energy costs, but also occupy limited on-chip NoC bandwidth. Both factors are proportional to the total NoC distance from the producer and consumer layers to the buffer, so reducing the total NoC distance can improve performance and energy efficiency.



Buffer Allocation Example ($3 \cdot c_5$ =300 KB, L = 50 KB)

Core	Remaining Capa.	Distance	Selected
1	200 KB	7	
2	40 KB	5	
3, 5	10 KB	7	
4, 6	10 KB	5	
7	150 KB	7	
8, 9	150 KB	5	√
10	0 KB	5	
11~16	30 KB	5	

Fig. 4: An Example of Mapping Six Layers on A 16-core Accelerator. In the table "Capa." stands for Capacity and "Distance" represents the Total NoC Distance. Cores are labeled from 1 to 16, in which core 8 and 9 are selected to buffer the 300KB requirement of $L5 \to L6$. L is set to 50KB.

Algorithm 1: Buffer Allocator

Input: List of Requirements req_list, Remain Space of Each Buffer $rems$, Size Limit of Each Allocation L
Output: Buffer Allocated to Each Requirement $alloc$

```
// Iteratively allocate each requirement req
for each req in req_list do
    // Filter all buffers with at least L remaining capacity
    avail_bufs = [x for x in rems if rems[x] ≥ L]
    // Then sort with total NoC distance and allocate in this order
    sort_key = lambda i: dist(req.src, i) + dist(i, req.dst)
    avail_bufs.sort(key = sort_key)
    remain_req = req.size
    for each buf in avail_bufs do
        // Allocate all remaining capacity of buf to req
        alloc_size = min(remain_req, rems[buf])
        remain_req -= alloc_size
        rems[buf] -= alloc_size
        alloc[req][buf] = alloc_size
        if remain_req == 0 then
            break
        end
    end
    if remain_req > 0 then
        // All avail_bufs has used up, must use DDR
        alloc[req][DDR] = remain_req
    end
end
return alloc
```

Allocated Size of Each Buffer: If the allocated size on a buffer is too small (e.g., less than 1KB), the buffer will suffer fragmentation issues, and transmitting such a small amount of data is not NoC-friendly either.

Considering all three factors above, the Buffer Allocator is implemented with a greedy algorithm (Alg. 1): For each buffer requirement, we set a limit L, which is a configurable parameter, for the smallest allocated size on each buffer, and sort all cores with enough remaining buffer capacity by their total NoC distance to the producer and consumer layers. Then we allocate from the nearest core to the furthest, allocating all remaining capacity of its buffer, until the requirement is satisfied. If all such buffers cannot satisfy the requirement, DRAM is used to buffer the unsatisfied part. In our experiments, L is set to one-sixth of the total required size based on our experiment observations. In this way, the number of cores to buffer will not exceed 6, and the allocated size of each buffer will not be too small; on top of that, the Buffer Allocator minimizes total NoC distance, taking great care of all three factors.

Fig. 4 shows an example of buffer allocation: The edge $L_5 \to L_6$ has a buffer requirement of 3 subbatches, which is 300KB. L is set to 50KB, and core 1,3,4,5,6 have limited unused buffer capacity that does not exceed L. Among all the cores that has enough capacity, core 8 and 9 has minimal total NoC distance, thus is prioritized to satisfy the requirement. Although all cores of L_3 have minimal total NoC distance, their remaining capacity is too small (30KB) and we do not utilize such core in consideration of fragmentation.

C. BufferProspector Implementation

We have developed an end-to-end compiler framework based on BufferProspector for our test chip. Except for BufferProspector Scheduling Part, the compiler includes a model parser that processes models from mainstream frameworks like PyTorch or TensorFlow

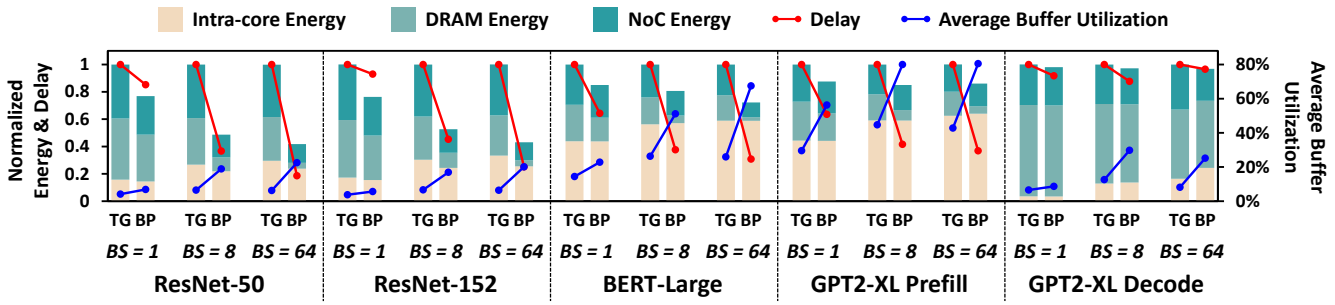


Fig. 5: Overall Comparisons Between Tangram (TG) and BufferProspector (BP). $BS = N$ is short for "batch size is N". Energy and Delay are normalized to Tangram in each pair of columns, while Average Buffer Utilization are shown in absolute values.

into the format required by our system. It also features an Intermediate Representation (IR) generator that organizes the information from the explored mapping scheme to create an IR conducive to instruction generation. Finally, an instruction generator transforms the IR into deployable hardware instructions.

V. Evaluation

A. Experiment Setup

1) Hardware Configuration: The default hardware architecture consists of 12×10 cores (this scale of core count is referenced from the well-known Tenstorrent Grayskull [5]), with each core equipped with a 1MB GBUF and 1024 MACs. The total DRAM bandwidth is set at 0.5GB/s per TOPs (120GB/s for the 120-core default accelerator). We also conduct sensitivity tests on accelerators with different scales. To systematically quantify the energy consumption for distinct operations required by the Tangram energy evaluators, such as MAC and buffer accesses, we employ a diverse array of tools and data. This includes directly obtaining data from commercial IP datasheets, or simulate RTL codes through a simulation flow developed during the chip design.

2) Workloads: In the overall comparison, we select ResNet-50 [1], ResNet-152 [1], BERT-Large [19], and GPT2-XL [20] as workloads due to their popularity in image and language applications and diverse structures; batch size is scaled among 1, 8 and 64, targeting both throughput-driven and latency-driven scenarios and demonstrating the efficiency of BufferProspector across different batches.

For the Design Space Exploration (DSE) experiments, we select BERT-Large as the default workloads regarding to its wide application; to include both cloud-based, throughput-driven scenarios and edge-based, latency-driven scenarios, batch size of 1 and 64 are selected; we also scale buffer size and DRAM bandwidth to study their impact on the effect of BufferProspector.

3) Baseline: We choose the open-sourced SOTA Tangram scheduling framework [11] as our baseline, since it employs optimized pipeline searching algorithms and mainstream intra-core tiling methods. The hardware configurations of Tangram and BufferProspector are kept the same throughout all experiments. Energy-delay Product (EDP) is selected as the optimization goal in all experiments, which matches the original Tangram paper [11].

B. Overall Comparison

As shown in Fig. 5, compared to Tangram, BufferProspector has demonstrated its value by achieving an improvement of 2.26× performance and 1.44× energy efficiency, on average across the five networks. A notable average reduction of 47.3% in DRAM energy consumption is observed, serving as the primary driver behind the observed enhancements in both performance and energy efficiency. This reduction can be primarily attributed to an increased buffer utilization, averaging 2.26 times, facilitating greater on-chip data reuse. The augmentation in buffer utilization can be attributed to two main advantages brought about by the BufferProspector strategy: (1) It relaxes the constraints on pipeline length present in Tangram (as introduced in Sec. II-B2, Tangram forfeits the mapping of layers that span more than one block due to timing mismatch challenges), allowing more layers to be computed on-chip concurrently, and enabling more feature maps to be directly generated and consumed on-chip without the need for DRAM access. (2) Feature maps that cannot be immediately consumed are temporarily stored on-chip, which avoids the need to write to DRAM and prevents stalling the overall pipeline.

1) Comparison among Networks: BufferProspector exhibits a significant performance improvement in all 6 workloads, which demonstrates the universal improvement of BufferProspector under different network topologies. The only exception here is the decode of GPT2-XL: GPT2-XL Decode has low compute density [21], and the dependency data for on-chip reuse is significantly smaller compared to the weight and KV cache data, leaving limited potential for BufferProspector to optimize.

2) Comparison among Batch Sizes: BufferProspector consistently achieves improvements over Tangram across various batch sizes. Even in latency-sensitive scenarios, where batch sizes can be as small as one, BufferProspector improves 1.29× performance and 1.19× energy efficiency compared to Tangram. The benefits of BufferProspector become increasingly pronounced with larger batch sizes, yielding 3.27× performance improvement and 1.66× energy efficiency improvement at batch size 64. This enhancement primarily attributes to the mitigation of filling and draining overheads in larger batch sizes. Since overheads brought by filling and draining is only relatively proportional to pipeline depth and remains the same in different batch sizes, lengthy pipelines will suffer less from such overheads in larger batch sizes. As a result, longer pipelines can be more efficiently mapped onto the accelerator, maximizing the advantages of BufferProspector and concurrently highlighting the constraints Tangram imposes on pipeline depth. As evidence, we observe that with the increase in batch size, BufferProspector reduces more DRAM access and enhances better buffer utilization. This indicates the ability of BufferProspector to buffer early-produced fmaps and utilize longer pipeline, thereby reducing DRAM access.

C. Design Space Exploration

To further investigate the architecture and layer-pipeline mapping of many-core accelerators, we conduct a DSE, providing in-depth analysis and discussion on the two critical factors of mapping and architecture: pipeline depth and buffer size.

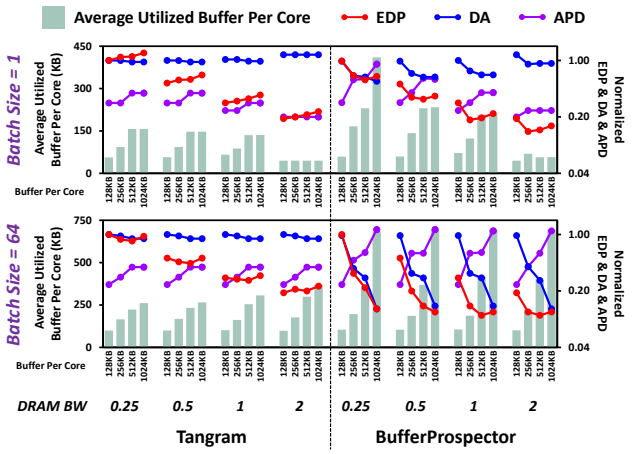


Fig. 6: DSE Experiments. The workload is Bert_Large. **DRAM BW** stands for DRAM bandwidth and 0.25 means "a DRAM bandwidth of 0.25GB/s per core", **DA** stands for DRAM Access, and **APD** stands for Average Pipeline Depth. EDP and DA in each plot is normalized to the first data point of Tangram, i.e. (128KB, 0.25GB/s), and APD is divided by 10 to match the scale. Normalized EDP, DA and APD is drawn in log-scale for better comparison.

1) Will A Deeper Pipeline Be Better? To analyze the influence of pipeline depth on overall performance, it is essential to evaluate the trade-offs associated with a deeper pipeline. On one hand, a deeper pipeline reduces the number of fmaps that need to be saved to DRAM, thereby lowering DRAM access and saving both energy and latency. On the other hand, it increases filling and draining overhead due to dependencies, as well as bubble costs caused by computational imbalances among different layers, which decreases computational utilization and negatively impacts performance. Therefore, determining whether a deeper pipeline is advantageous hinges on assessing whether the benefits outweigh the drawbacks, a trade-off that varies depending on the specific scenario.

When batch size is large, filling & draining overheads are relatively small, and its disadvantage is covered by the benefits of DRAM access reduction. Thus extending the pipeline is beneficial in most times, and both Tangram and BufferProspector will try their best to extend the pipeline depth, hoping for more DRAM access reductions. However, in a deeper pipeline, more layers are mapped onto the cores at the same time, posing a much larger buffer requirement. Thus, for BufferProspector, it will try to make full use of the buffer to pursue a deeper pipeline. This can be evidenced by the huge increase in APD and utilized buffer in Fig. 6 batch size 64, where the reduction of DRAM Access matches the increase of APD. Although Tangram will also try to extend its pipeline, it suffers from timing mismatch issues (introduced in Sec. II-B2) and has a much smaller APD limit, so APD will remain the same after hitting this limit, even with larger buffer, which greatly restricts its performance improvements. This can be seen clearly in Fig. 6 batch size 64.

When batch size is small, the pipeline will experience a much larger filling & draining overheads, thus compared with large batch sizes, extending the pipeline becomes less beneficial and its downside becomes comparable with the improvement of DRAM access reductions. As a result, pipeline tends to be longer when DRAM bandwidth is less sufficient (see Fig. 6 batch 1). Under a smaller DRAM bandwidth, the whole computation becomes more bounded by DRAM accesses, which highlights the contribution of DRAM access reductions to the reduction of overall latency, making longer pipelines more beneficial, and encouraging both Tangram and BufferProspector to extend to a larger pipeline, as shown in Fig. 6 batch 1.

In summary, in large-batch-size scenarios, BufferProspector will try its best to extend its pipeline, and pipeline depth is mostly constrained by the buffer size; while in small-batch-size scenarios, DRAM bandwidth will also be a decisive factor, and a smaller DRAM bandwidth prefers a longer pipeline. **Thus for BufferProspector, one should focus on designing larger buffers in throughput-driven scenarios and focus**

2) Will A Larger Buffer Be Better? When buffer size per core increases from 128KB to 1024KB, BufferProspector on average can reduce EDP by 47.4%. Such huge improvement mainly stems from the possibility of buffering more fmaps on-chip and reducing more DRAM access, as evidenced by a 5.21x average increment of utilized buffer and a 60.5% average reduction of DRAM access.

However, we can observe that such reduction of EDP becomes smaller when buffer size is larger, and for the 1GB/s and 2GB/s settings, the EDP forms a U-shaped curve, even increasing when buffer per core increases from 512KB to 1024KB. There are two main reasons that lead to the smaller reduction, or even increase, of EDP. Firstly, the pipeline cannot extend infinity, and fmaps that needs to be buffered is also limited, thus when the buffer is larger, fmaps that needs to be optimized to on-chip buffer is also fewer, which means expanding it brings less benefit on the EDP. In addition, accessing a larger buffer needs a larger energy cost. For example, it is shown that when the size of a buffer doubles, its access energy cost becomes 1.5 times larger [22]. Thus, an increased buffer size results in a larger energy cost, leading to an increased EDP.

In conclusion, on the one hand, larger buffer usually means better performance; but on the other hand, when buffer becomes larger, the reduction of DRAM access becomes less prominent, while the increase of energy cost becomes more dominant, giving out less performance optimizations, and when the extra energy cost outruns the reduced access, we will even get worse performance. **Thus to pursue the best performance of BufferProspector, one should not aggressively increase the buffer size, but find and follow the appropriate buffer size in accelerator designs.**

Notice that Tangram can barely reduce DRAM access with a larger buffer, thus suffering from an average increase of 14.5% in EDP when buffer size per core increases from 128KB to 1024KB.

VI. Conclusion

This paper begins by highlighting a deficiency commonly observed in existing LP mapping strategies: severe under-utilization of many cores' buffers. We then conduct qualitative and quantitative analyses of this phenomenon, deducing that under current LP mapping strategies, a linear relationship exists between a layer's DCR attribute and buffer utilization. This establishes the phenomenon's hardware independence and widespread prevalence at a theoretical level. Following this, we develop the BufferProspector strategy, which includes a Buffer Requirement Calculator applicable to any DAG and an efficient Buffer Allocator. These tools aim to leverage the substantial amounts of underused on-chip buffer resources discovered by us to address the timing mismatch challenges in layer-pipeline mapping. Finally, our empirical experiments demonstrate that BufferProspector offers significant performance and energy efficiency improvements over the SOTA open-source LP mapping framework, Tangram.

VII. Acknowledgment

This work is supported by Beijing Municipal Science and Technology Program under Grant No.Z241100004224028.

REFERENCES

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR 2016*, pages 770–778. IEEE Computer Society, 2016.

[2] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR 2017*, pages 2261–2269. IEEE Computer Society, 2017.

[3] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A. Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. In Satinder P. Singh and Shaul Markovitch, editors, *AAAI 2017*, pages 4278–4284. AAAI Press, 2017.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS 2017*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc.

[5] Jasmina Vasiljevic, Ljubisa Bajic, Davor Capalija, Stanislav Sokorac, Dragoljub Ignjatovic, Lejla Bajic, Milos Trajkovic, Ivan Hamer, Ivan Matosevic, Aleksandar Cejkov, Utku Aydonat, Tony Zhou, Syed Zohaib Gilani, Armond Paiva, Joseph Chu, Djordje Maksimovic, Stephen Alexander Chin, Zahi Moudallal, Akhmed Rakhmati, Sean Nijjar, Almeet Bhullar, Boris Drazic, Charles Lee, James Sun, Kei-Ming Kwong, James Connolly, Miles Dooley, Hassan Farooq, Joy Yu Ting Chen, Matthew Walker, Keivan Dabiri, Kyle Mabee, Rakesh Shaji Lal, Namal Rajatheva, Renjith Retnamma, Shripad Karodi, Daniel Rosen, Emilio Munoz, Andrew Lewycky, Aleksandar Knezevic, Raymond Kim, Allan Rui, Alexander Drouillard, and David Thompson. Compute substrate for software 2.0. *IEEE Micro*, 41(2):50–55, 2021.

[6] Sean Lie. Multi-million core, multi-wafer AI cluster. In *HCS 2021*, pages 1–41. IEEE, 2021.

[7] Emil Talpes, Douglas Williams, and Debjit Das Sarma. DOJO: the microarchitecture of tesla's exa-scale computer. In *HCS 2022*, pages 1–28. IEEE, 2022.

[8] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel Emer, C. Thomas Gray, Brucek Khailany, and Stephen W. Keckler. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *MICRO 2019*, MICRO '52, page 14–27, New York, NY, USA, 2019. ACM.

[9] Jingwei Cai, Xuan Wang, Mingyu Gao, Sen Peng, Zijian Zhu, Yuchen Wei, Zuotong Wu, and Kaisheng Ma. Soma: Identifying, exploring, and understanding the dram communication scheduling space for dnn accelerators. *arXiv preprint arXiv:2501.12634*, 2025.

[10] Jingwei Cai, Yuchen Wei, Zuotong Wu, Sen Peng, and Kaisheng Ma. Inter-layer scheduling space definition and exploration for tiled accelerators. In Yan Solihin and Mark A. Heinrich, editors, *ISCA 2023*, pages 13:1–13:17. ACM, 2023.

[11] Mingyu Gao, Xuan Yang, Jing Pu, Mark Horowitz, and Christos Kozyrakis. TANGRAM: optimized coarse-grained dataflow for scalable NN accelerators. In Iris Bahar, Maurice Herlihy, Emmett Witchel, and Alvin R. Lebeck, editors, *ASPLOS 2019*, pages 807–820. ACM, 2019.

[12] Jingwei Cai, Zuotong Wu, Sen Peng, Yuchen Wei, Zhanhong Tan, Guiming Shi, Mingyu Gao, and Kaisheng Ma. Gemini: Mapping and architecture co-exploration for large-scale dnn chiplet accelerators. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 156–171. IEEE, 2024.

[13] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel S. Emer. Timeloop: A systematic approach to DNN accelerator evaluation. In *ISPASS 2019*, pages 304–315. IEEE, 2019.

[14] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach. In *MICRO 2019*, pages 754–768. ACM, 2019.

[15] Open-sourced framework of bufferprospector. https://github.com/SET-Scheduling-Project/BufferProspector-DAC2025.

[16] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *CVPR 2015*, pages 1–9. IEEE Computer Society, 2015.

[17] Xinhan Lin, Shouyi Yin, Fengbin Tu, Leibo Liu, Xiangyu Li, and Shaojun Wei. LCP: a layer clusters paralleling mapping method for accelerating inception and residual networks on FPGA. In *DAC 2018*, pages 16:1–16:6. ACM, 2018.

[18] Xingbin Wang, Boyan Zhao, Rui Hou, Amro Awad, Zhihong Tian, and Dan Meng. Nasguard: A novel accelerator architecture for robust neural architecture search (NAS) networks. In *ISCA 2021*, pages 776–789. IEEE, 2021.

[19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019.

[20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[21] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[22] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, Ankita Nayak, Steven Bell, Kaidi Cao, Heonjae Ha, Priyanka Raina, Christos Kozyrakis, and Mark Horowitz. Interstellar: Using halide's scheduling language to analyze DNN accelerators. In James R. Larus, Luis Ceze, and Karin Strauss, editors, *ASPLOS '20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*, pages 369–383. ACM, 2020.